Bilateral Receipt Admission: Cross-Organizational Action Provenance with Treaty-Bound DSSE

Anonymous Author(s)

Abstract

Existing supply-chain provenance attests how an artifact was built; cross-organizational agent invocations need provenance for how an action was admitted at the receiving kernel. SLSA, in-toto, Sigstore, and Rekor describe production; none attest that two parties jointly admitted an action under a treaty-bound predicate over identical canonical bytes including a continuation that links to receipt-graph state. We define a DSSE predicate type whose subject digest binds a tuple of treaty scope, ladder-intersection hash, request and outcome digests, continuation hash, and two receipt hashes under two named kernel keys, and a strict verifier with five rejection codes that distinguish noncanonical payloads, predicate-type mismatch, signer reuse, stale leases, and subject-digest mismatch. We implement the construction as a Rust runtime with a pre-dispatch admission hook that fails closed on federated-origin requests, with selective disclosure via BBS at presentation while Ed25519 remains authoritative for audit corpora. We demonstrate the construction with a three-vendor buyer-closure that replays both admitted and denied paths in the same canonical schema. The construction defeats sibling-treaty cross-receipt substitution, BBS stub-vs-real disambiguation, and error-message oracles by construction; the companion position paper defends the full polity, amendment, and Lean-attestable construction.

1 Introduction

Tool-using agents already behave like institutional actors, receiving delegated authority, calling tools across organizational boundaries, and emitting records that become accounting evidence. The weak point is provenance for cross-vendor action: existing supply-chain provenance (SLSA, in-toto, Sigstore, Rekor) attests how an artifact was *built*; agent invocations need provenance for how an action was *admitted at the receiving kernel*. No deployed primitive attests that two parties jointly admitted an action under a treaty-bound predicate over identical canonical bytes that include a continuation linking to receipt-graph state.

We close that gap with a single cryptographic construction. A bilateral DSSE envelope of predicate type `chio.bilateral-cosign-invocation.v1` carries a subject digest computed over the canonical receipt body that binds treaty-scope hash, ladder-intersection hash, request and outcome digests, continuation hash, and the two participating kernels' receipt hashes. A strict verifier accepts the envelope iff (i) both signatures verify against signer keys that are independent and pinned in the verifier-owned trust store; (ii) the payload is canonical JSON; (iii) the statement type is in-toto and the predicate type is the strict Chiodos profile; (iv) the leases referenced by the payload are fresh against the verifier's revocation epoch; and (v) the subject digest equals the canonical hash of the bound tuple. Five rejection codes distinguish each failure class so downstream audit can identify which constitutional precondition failed.

This paper defends the bilateral-DSSE primitive on its own technical merits. A companion paper extends the construction to a polity model with Lean-attestable amendments, treaty-of-treaties composition, and political-theory framing. The present paper deliberately scopes to the cryptographic and runtime primitives that survive without that framing.

The contributions are four falsifiable artifacts:

- A DSSE predicate type and canonical subject-digest binding for bilateral cross-kernel admission, with a five-code rejection taxonomy.
- A Rust runtime with a pre-dispatch admission hook that fails closed on federated-origin requests, plus a strict bilateral verifier.
- A three-vendor buyer-closure that replays admitted and denied paths in the same canonical schema.
- A negative-result characterization: sibling-treaty cross-receipt substitution, BBS stub-vs-real disambiguation, single-lane witness compromise, and error-message oracles are rejected by construction.

2 Receipt Admission as a Primitive

Signed-evidence systems decide one of two questions. Supply-chain transparency logs (Certificate Transparency, Rekor, in-toto, SLSA) answer how an artifact was built and whether a build-time claim was published to a public log [12, 18, 20–22]. Authorization systems (Cedar, OPA, capability OSes) answer whether a principal may perform an action inside a single trust boundary [6, 7, 11, 14]. Neither addresses the question agent systems force: whether two organizations jointly admitted a cross-vendor action under predicates each holds independently, in a form a third-party verifier can replay over canonical bytes.

The bilateral-receipt-admission primitive is the smallest construction that answers that question. It separates four concerns:

Local enforcement. Each kernel evaluates a local predicate set against a tool-call request. The predicate set may include capability attenuation [14], information-flow gates [23], or arbitrary local policy. The local decision is recorded in a canonical receipt body signed with the kernel's Ed25519 key.

Treaty-bound co-signing. When the action crosses an organizational boundary, both kernels independently sign a DSSE envelope whose subject digest binds the two receipts plus the treaty's scope, ladder, and continuation state. The envelope is admissible only when both signatures verify, the predicate type matches, and the canonical bytes are identical at both signers.

Strict rejection. A verifier accepts only envelopes that satisfy every precondition. The rejection codes are part of the protocol so downstream audit can identify the failing precondition.

Selective disclosure at presentation. BBS group signatures [3, 8, 13] project the receipt body for presentation to a third party (regulator, auditor, counterparty) without exposing every field. Ed25519 over canonical bytes remains authoritative for audit corpora; BBS is presentation-only.

The four concerns compose: local enforcement is independent of the cosigning protocol; cosigning is independent of selective disclosure; the verifier's rejection codes are independent of all three. A deployment can adopt them incrementally.

3 Predicate Schema and Strict Verifier

The bilateral admission envelope is a DSSE statement [18] carrying the predicate type `chio.bilateral-cosign-invocation.v1`. The payload type is the in-toto statement type, the statement type is in-toto v1, and the predicate body is a single record over canonical JSON [15] that binds, in one tuple: treaty id, treaty-scope hash, ladder-intersection hash, admission-report hash, continuation hash, request-canonical hash, outcome-canonical hash, local-receipt hash, remote-receipt hash, capability lease, governance receipt reference, and the two signer kernel ids. The DSSE subject is a single digest whose preimage is the canonical hash of that tuple; the envelope carries exactly two signature slices, one per kernel keyid.

This shape differs structurally from generic DSSE provenance. SLSA provenance and in-toto link statements [21, 22] bind a build pipeline to an artifact; their subject is the artifact digest and the predicate records build parameters. The bilateral profile binds a receipt-graph admission event to a treaty, with the subject digest taken over a tuple of hashes rather than an artifact, and with joint authorization intent expressed as two independent kernel signatures over identical canonical bytes. Without that intent record a receiver can verify that some party signed something, but cannot distinguish a shared action from two adjacent local logs that happen to name the same workflow.

Verifier accept set. Let the trust store $\mathcal{S} = (\mathit{IssuerKeys}, \mathit{KernelKeys}, \mathit{LeaseEpoch})$ and let E be a bilateral envelope. The verifier accepts iff

$$
\begin{aligned}
\mathit{accept}(\mathcal{S}, E) \iff\ & \mathit{canonical}(E.\mathit{payload}) \wedge \\
& E.\mathit{predicateType} = \mathit{ChiodosBilateralV1} \wedge \\
& |E.\mathit{sigs}| = 2 \wedge \\
& E.\mathit{sig}_1.\mathit{kid} \neq E.\mathit{sig}_2.\mathit{kid} \wedge \\
& E.\mathit{sig}_1.\mathit{kid} \in \mathit{IssuerKeys} \wedge \\
& E.\mathit{sig}_2.\mathit{kid} \in \mathit{KernelKeys} \wedge \\
& E.\mathit{payload.lease.epoch} \geq \mathcal{S}.\mathit{LeaseEpoch} \wedge \\
& E.\mathit{subjectDigest} = \\
& \quad H(\mathit{canonical}(E.\mathit{payload.bindingTuple})).
\end{aligned}
$$

Each verified signature is over the same canonical payload bytes. The predicate is the conjunction of six gates, in that order; the verifier evaluates them left-to-right and returns one of five rejection codes at the first failure, or admits the envelope.

noncanonical-payload. The payload bytes fail RFC 8785 canonicalization: re-ordered keys, non-minimal numeric forms, unnormalized strings, or trailing whitespace. The verifier rejects rather than re-canonicalizing because the signature is over a specific byte sequence; an envelope that signs noncanonical bytes admits a presentation attacker who can serve different bytes to different verifiers without breaking any signature. The check protects the canonical-bytes assumption that downstream selective disclosure and replay both depend on.

predicate-type-mismatch. The DSSE statement carries a `predicateType` other than `chio.bilateral-cosign-invocation.v1`. Generic in-toto profiles do not bind admission-report and ladder-intersection hashes; treating an SLSA build provenance, a generic SCAI predicate, or a degenerate signed-JSON record as bilateral admission would let a sender substitute a weaker predicate and still pass syntactic envelope checks. Pinning the predicate string makes profile downgrade impossible without forging a signature.

signer-reuse. The two signature slices carry the same keyid, or distinct keyids that resolve to the same operator under verifier-owned attribution metadata. A single signer cannot produce two-party consent; party-independence is what distinguishes bilateral admission from a single kernel signing twice. The keyid-equality test is local; deeper operator-independence checks live in the trust store and depend on attestation evidence the verifier accepts out of band.

stale-lease. The capability lease referenced by the payload predates the verifier's current revocation epoch, or its expiry has passed. The lease binds the action's authority window; admitting a stale lease re-admits a window the issuer has already revoked, and turns the receipt graph into an append-anything store. Lease freshness is a verifier-side test against state the verifier owns, not a sender-asserted claim.

subject-digest-mismatch. The DSSE subject digest is not the canonical hash of the binding tuple constructed from the payload's named fields. This is the integrity check on the binding itself: a payload whose treaty-scope hash, ladder-intersection hash, request hash, outcome hash, and the two receipt hashes disagree with the digest the signers committed to is structurally not the envelope the signers intended. The mismatch also defeats sibling-treaty substitution, since a receipt issued under one treaty's ladder-intersection cannot satisfy another treaty's subject digest without colliding the hash.

The taxonomy is intentionally coarse. Each code names a conjunct of the admission predicate, not a structured failure report identifying which sub-field disagreed at the bit level. An adversary probing the verifier learns which gate first refused, not which inner field mismatched; the error surface leaks at most $\log_2 5 \approx 2.32$ bits per attempt, and that information is exactly what an auditor needs to identify which constitutional precondition failed.

4 Formal Sketch

The verifier of §3 has a freestanding structural theorem at the primitive layer. Strip the runtime to the three load-bearing structures: a trust store $\mathcal{S}$ with separate issuer-key and kernel-key allowlists, a bilateral envelope E carrying a receipt id, a scope predicate, and two signature slices (one issuer-side, one kernel-side), and a denotational interpreter $[\![\cdot]\!]$ for the scope predicate against the receipt

id. The accept relation is the conjunction

$$\begin{aligned} accept(\mathcal{S}, E) \;&= [E.issuerSig.kid \in \mathcal{S}.issuerKeys] \\ &\wedge [E.kernelSig.kid \in \mathcal{S}.kernelKeys] \\ &\wedge [\![E.scope]\!](E.receiptId). \end{aligned}$$

Signature byte validity is abstracted into trust-store membership: the trust store names keys whose signatures the deployed verifier has separately checked, and the freestanding theorem inspects only the membership and scope conjuncts.

Theorem 4.1 (Freestanding accept set). *For every trust store $\mathcal{S}$ and bilateral envelope E,*

$$\begin{aligned} accept(\mathcal{S}, E) \iff\; & E.issuerSig.kid \in \mathcal{S}.issuerKeys \\ &\wedge E.kernelSig.kid \in \mathcal{S}.kernelKeys \\ &\wedge [\![E.scope]\!](E.receiptId) = true. \end{aligned}$$

The bi-conditional is the load-bearing claim. The forward direction says the verifier returns true only when all three conjuncts hold; the reverse direction says no hidden side channel can sneak admission past any of the three. The proof is one Lean tactic line that unfolds the definition of *accept* and rewrites the resulting Boolean conjunction. The Lean source is self-contained at the module level: it imports the predicate-language type and its denotational interpreter, but takes no dependency on the wider polity model, anchor-quorum machinery, or treaty-intersection construction. A reviewer can audit the file end-to-end at `formal/lean4/Chio/Chio/Treaty/BilateralAccept.lean`.

Two corollaries follow definitionally. Accept-set monotonicity in the trust store says extending the issuer or kernel allowlist preserves prior accepts: adding a co-signing counterparty never invalidates an envelope that admitted under the smaller store. Accept-set conjunction over scope predicates says an envelope accepted under scope $p \wedge q$ remains accepted when its scope is rebound to p alone or q alone; this is the structural decomposition that lets a verifier evaluate joint admission against a treaty as the intersection of independently named predicates. Both corollaries are proved by unfolding *accept* and rewriting on the freestanding theorem.

The theorem deliberately stops short of three further obligations. It does not prove single-amendment backward refinement: the freestanding statement is over a fixed predicate language, not over amendments that propose a new predicate set, and the trajectory invariant that rules out constitutional ratchets is not in scope at the primitive layer. It does not prove polity-level admission: it speaks only of the verifier's accept set on a bilateral envelope, not of a polity's constitutional admission of a receipt against its own predicate set and citizenship roster. And it does not address the Hart-condition-(a) framing for state-like authority: the construction is a cryptographic primitive over canonical bytes, and the political-theory framing belongs to a different argument.

Those three obligations are discharged separately in the companion polity paper: `amendment_admissible_iff_backward_refinement` carries amendment refinement as a type-level invariant on the enactment constructor; `treaty_admission_iff_predicate_intersection` carries treaty admission as the Boolean intersection of treaty scope, treaty constitution, and both polities' *polityAdmits* relations; and the trajectory-invariant theorem `essential_preserved_chain` rules out a chain of individually valid refinements that collapse to an adversary-only predicate over admitted history. The bilateral-DSSE primitive sits below all three: it is the byte-level admission gate the polity model depends on, with its own structural correctness theorem that does not borrow polity machinery.

5 Implementation

The primitive lives in three Rust components: a runtime kernel that exposes a pre-dispatch admission hook, a federation crate that implements the strict bilateral DSSE verifier, and a selective-disclosure module that projects canonical receipt bytes into a BBS message vector for presentation. The wire format is the load-bearing artifact; the implementation is one realization of it.

Pre-dispatch admission hook. The runtime exposes a single hook that intercepts every tool-call request before dispatch. Non-federated local requests use the existing local-policy path when no Chiodos admission context is present; once a Chiodos context appears, malformed context denies, and federated origin without treaty context denies with a named rejection code. The hook resolves the treaty reference from the call site rather than from request metadata: request-level smuggling of trust roots, peer directories, signing keys, or dynamic trust bundles is rejected by construction, because the resolver consults only verifier-owned state. Treaty DSSE evidence is verified against verifier-owned trust store artifacts; signer sets must equal the treaty's declared participants and public keys must be independent. The hook releases reserved continuation state on denial or abort, closing the replay edge that an attacker would otherwise exploit by re-submitting a continuation that was admitted-then-aborted.

The asymmetry is deliberate. Allowing federated origin without treaty context would make the bilateral DSSE primitive irrelevant to cross-kernel execution. Allowing request metadata to smuggle a trust root would collapse the verifier-owned-store assumption that the freestanding theorem of §4 rests on. Failing to release reserved continuation state on denial would create a state leak and an ambiguous replay surface. Each is a concrete place where agent middleware turns formal policy into advisory logging; the hook refuses each pattern.

Strict bilateral verifier. The federation crate at `crates/chio-federation/src/bilateral_dsse.rs` defines the strict predicate and envelope. The verifier function `verify_chiodos_dsse_envelope` implements the five-gate check of §3: it rejects wrong payload type, wrong signature count, noncanonical JSON, wrong statement type, wrong predicate type, missing subject, and reused signer keys. A companion operational verifier at `crates/chio-federation/src/bilateral_verifier.rs` layers checks the freestanding theorem treats as trust-store assumptions: peer key pins, fresh ladder manifests, revocation epoch, receipt resolution and signature, tool-argument hash, policy-verdict agreement, capability lease, governance receipt for receipt-backed action classes, and the treaty binding references. Together, the envelope check and the operational check ensure admission depends on the predicate the protocol names, not on a strict subset a sender could satisfy with degenerate data.

A DSSE envelope with the wrong predicate is not a near miss; a payload signed by the same key twice is not two-party consent;

a receipt with a valid signature but the wrong request hash is not admissible under the treaty; receipt-backed classes require a governance receipt even when the ordinary receipt is otherwise valid. The five-code error taxonomy is part of the protocol because downstream audit must know which constitutional precondition failed.

Selective disclosure. The selective-disclosure module signs a BBS projection of the canonical receipt body and verifies derived disclosure proofs against an issuer-owned key registry [3, 13]. The Ed25519 signature over canonical receipt bytes remains authoritative for audit corpora; BBS is a presentation-only commitment that lets a holder reveal a subset of receipt fields to a verifier without exposing every field. The verifier pins the issuer key and the projection version, so the proof is not free-form redaction.

Stub disclosure. One platform-specific stub the primitive does not depend on is worth naming. The macOS Endpoint Security path used for kernel-side telemetry is a stub: the integration surface compiles and tests pass against canned fixtures, but live Endpoint Security event delivery is not wired up. The bilateral-DSSE primitive, the admission hook, the federation crate, and the selective-disclosure module operate on canonical bytes and trust-store state regardless of whether kernel-side telemetry is live; the stub is named so a deployer planning kernel-event-driven receipts knows where the gap sits.

6 Evaluation

The primitive is exercised on a three-vendor closure: a buyer, two vendors, and a single bilateral admission between the vendors. The buyer owns the verifier context and the proof package; vendor receipts are not imported merely because they are signed but only when they bind treaty scope, ladder intersection, continuation, lineage, bilateral invocation, and strict DSSE evidence to one canonical subject digest. The closure is a generator: every run emits both signed positive packages (admitted vendor-to-vendor actions) and signed negative packages (denied actions) in the same canonical schema, so the buyer can replay every admission decision and every denial without privileged access to vendor internals.

Admitted path. A vendor-to-vendor request resolves to a treaty τ whose scope, ladder, and participant manifests are pinned in the buyer's verifier-owned trust store. The vendor kernels jointly produce a bilateral DSSE envelope whose subject digest binds τ's scope hash, the ladder-intersection hash, the request and outcome canonical hashes, the continuation hash, and the two vendor receipt hashes; the envelope carries one signature per vendor over identical canonical bytes. The admission hook resolves the treaty from the call site, consumes the continuation before dispatch, evaluates the five-gate verifier, and admits the action. The receipt graph records one allow receipt per vendor plus the admission report; the buyer's negative-result audit later replays the same envelope against the same trust store and confirms admission.

Denied paths. Three denial fixtures cover the load-bearing rejection classes. (1) Sibling-treaty mismatch: a receipt issued under treaty T_A is replayed inside a request that resolves to sibling treaty T_B; the subject digest fails because T_A's scope hash and ladder-intersection hash differ from T_B's, and the verifier returns subject-digest-mismatch. (2) Signer reuse: both DSSE signature slices carry the same vendor keyid, which fails the party-independence gate even though both signatures verify against pinned keys. (3) Stale lease: the capability lease referenced by the payload predates the buyer's revocation epoch; the lease-freshness gate fails before the digest gate is reached. In each case the negative-result package is signed, canonical, and shaped identically to a positive package: a downstream reviewer replays the canonical bytes and reconstructs both the admission attempt and the rejection code.

Dispatch latency. The pre-dispatch admission hook is reused from the production Criterion bench for the allow path. On the baseline machine (Apple M1 Max, 10 cores, 32 GB RAM, Darwin 25.4.0) the median dispatch-admission time is p50 72.051 μs (Criterion 95% CI 71.745 μs to 72.367 μs), where the body exercises the local-policy path that the federated admission hook composes with. The measurement is reported as Criterion p50 with a 95% confidence interval; the bilateral DSSE verification adds a constant cost on top of this path (two Ed25519 verifies plus a SHA-256 over the canonical binding tuple), which dominates only when the kernel's local policy is trivially cheap. Receipt sign and receipt verify latencies are out of scope: their benches black-box constants in the current harness, and reporting them would measure Criterion overhead rather than the primitive.

Replay corpus stability. The replay gate runs a kernel-capability golden target across 50 fixtures spanning ten capability-side families (allow simple, allow metered, allow with delegation; deny expired, deny revoked, deny scope mismatch; guard rewrite; replay attack; tampered canonical JSON; tampered signature). Each fixture's expected verdict is canonicalized against a stored manifest; the reported property is manifest stability across machines rather than re-execution of the kernel on adversarial input. The reported result is 50 fixtures, manifest-stable across machines (test wall-time 1.07 s). The 50-fixture corpus does not yet exercise the strict bilateral-DSSE rejection codes of §3; a Chiodos-specific replay corpus that covers noncanonical-payload, predicate-type-mismatch, signer-reuse, stale-lease, and subject-digest-mismatch fixtures is named as follow-up work alongside the limitations of §9.

What the closure demonstrates. Three properties survive without the polity-layer framing of the companion paper. First, admission is byte-level reproducible: the buyer replays the canonical envelope and recovers either the same admission or the same rejection code. Second, denial is auditable in the same format as admission: a signed negative package is not a degraded log entry but a first-class artifact with the rejection code as a structured field. Third, the verifier-owned trust store is the boundary the construction defends: the admitted path and the three denied paths differ only in whether the canonical subject digest, the signer set, and the lease epoch agree with the buyer's trust state, and every disagreement is named by exactly one rejection code.

7 Attacks Defeated by Construction

Five classes of attack against cross-organizational receipt evidence are rejected structurally rather than by detection. Each rejection

corresponds to a precondition the strict verifier checks before admission.

Sibling-treaty cross-receipt substitution. A receipt issued under treaty T_A cannot be replayed under sibling treaty T_B: the subject digest binds the treaty-scope hash and the ladder-intersection hash of T_A, and any verifier evaluating the envelope against T_B returns subject-digest-mismatch.

BBS stub-versus-real disambiguation. A presentation cannot fool a verifier into accepting a BBS proof that does not bind to the canonical bytes the Ed25519 receipt was signed over. The strict verifier treats Ed25519 as authoritative; BBS is a presentation-layer projection of the same message vector, verified against the disclosed-attribute subset of the canonical body. A BBS-only presentation without a matching Ed25519 record is not admissible.

Single-lane witness compromise. Anchor lanes (Rekor, OpenTimestamps, Solana memo, EVM event) have uneven maturity. A compromised or stub witness on one lane cannot promote a false receipt to admission: the predicate-type binding requires the treaty's declared multi-lane witness policy, evaluated over the receipt's full anchor set rather than any single lane's say-so.

Error-message oracles. The five rejection codes (noncanonical-payload, predicate-type-mismatch, signer-reuse, stale-lease, subject-digest-mismatch) are coarse: each names a conjunct of the admission predicate, not a structured failure report. An adversary learns which conjunct first failed, not which sub-field disagreed at bit level.

Constitutional ratchet at the policy layer. An attacker proposing an amendment that loosens admission for already-admitted history targets the predicate set itself rather than any individual envelope. This is out of scope here; the companion paper's trajectory-invariant theorem (`essential_preserved_chain`) obliges every amendment to refine the prior predicate set over admitted receipts.

8 Related Work

Supply-chain provenance. SLSA [21], Sigstore [20], in-toto [22], Rekor [19], and DSSE [18] describe build-time provenance: how an artifact was produced and by which pipeline. IRONDICT [9] extends transparent dictionaries with polynomial commitments, the direction Rekor-style append-only logs are converging toward. Bilateral receipt admission targets the adjacent but distinct question of runtime cross-organizational action provenance: not how an artifact was built, but how an action was admitted at the receiving kernel under treaty-bound predicates.

Property attestation. The closest formal lineage is property attestation: integrity-measurement architectures [17], semantic remote attestation [10], property-based attestation [16], and principles of remote attestation [5]. These attest properties of one system to a remote relying party; bilateral receipt admission attests that two systems jointly admitted an action over identical canonical bytes. Adjacent TEE-rooted attestation carries its own side-channel surface (UncoreBleed [4] on Intel SGX is a recent example) that complements rather than substitutes for canonical-bytes evidence.

Policy languages and verified authorization. Cedar establishes a Lean-plus-Rust pattern for proving policy decisions well-defined inside a single trust boundary [6, 7]. SAGA distributes access-control tokens from a user-controlled provider to constrain agent invocations [2]. Both work at the policy-decision layer inside one boundary; bilateral receipt admission proves a property at the cross-organizational admission layer, where the verifier replays a predicate over canonical bytes signed by two kernels. The layers compose.

Agent and tool isolation. IsolateGPT enforces per-app isolation with information-flow gates for LLM-based agents [23]; Omega proposes a TEE-rooted agentic runtime with declarative policy evaluation [1]. Both secure tool calls inside one operator's runtime; neither produces a cross-organizational artifact a third party can replay against a treaty predicate.

Companion paper. A companion paper situates the construction inside a polity model and formalizes the verifier's correctness as Lean theorems (`treaty_admission_iff_predicate_intersection`, `essential_preserved_chain`) with amendment as backward refinement. The present paper restricts to the primitives that survive without that framing.

9 Limitations

Three load-bearing limits sit at the primitive layer.

No live cross-process federation. The multi-party setup runs in-process: two kernels share a clock and exchange canonical bytes through library calls rather than the network. Cross-process federation and partition reconciliation sit in the companion paper's deployment trajectory.

Single-vendor key custody. Bilateral DSSE carries party-independence only when the two signing keys are held by independent operators. If both kernels run under one custody domain the construction reduces to single-key signing, and operational discipline rather than cryptography carries the independence claim. A kernel-independence attestation primitive is named as future work.

Observability gap. The receipt log is the audit channel: every admission and denial appears as a canonical receipt with a rejection code. Live observability (denial-rate alerts, schema-mismatch fingerprinting, non-leaking telemetry) is not specified.

Polity-level questions (citizenship, amendment, Hart-condition-(a) framing, trajectory-invariant theorems, sensor-state attestation) sit one layer above this primitive and are addressed in the companion paper.

References

[1] Anonymous. 2025. Omega: A TEE-Rooted Agentic Runtime with Declarative Policy Evaluation. arXiv preprint. https://arxiv.org/html/2605.03213

[2] Anonymous (NDSS 2025). 2025. SAGA: A Security Architecture for Governing AI Agentic Systems. In *32nd Network and Distributed System Security Symposium (NDSS)*. https://www.ndss-symposium.org/ndss-paper/saga-a-security-architecture-for-governing-ai-agentic-systems/

[3] Dan Boneh, Xavier Boyen, and Hovav Shacham. 2004. Short Group Signatures. In *Advances in Cryptology - CRYPTO 2004*. 41–55. doi:10.1007/978-3-540-28628-8_3

[4] Decheng Chen, Zhi Zhang, Zhenkai Zhang, Xin Zhang, Yansong Gao, and Yi Zou. 2026. UncoreBleed: AEX-Free, High-Resolution, and Low-Noise Side-Channel Attacks on SGX Enclaved Execution. In *Proceedings of the 35th USENIX Security Symposium (Cycle 1)*. USENIX Association, Berkeley, CA, USA.

[5] George Coker, Joshua Guttman, Peter Loscocco, Amy Herzog, Jonathan Millen, Brian O'Hanlon, John Ramsdell, Ariel Segall, Justin Sheehy, and Brian Sniffen. 2011. Principles of Remote Attestation. *International Journal of Information Security* 10, 2 (2011), 63–81. doi:10.1007/s10207-011-0124-7

[6] Joseph W. Cutler, Craig Dill, Aaron Eline, Nate Foster, Karol Grosse, Mark McLaughlin, Neha Rungta, Caleb Stanford Smith, Serdar Tasiran, and Emina Torlak. 2024. Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization. In *Proceedings of the ACM on Programming Languages, OOPSLA*. https://2024.splashcon.org/details/splash-2024-oopsla/25/Cedar-A-New-Language-for-Expressive-Fast-Safe-and-Analyzable-Authorization

[7] Darin Cuvillier, Aaron Eline, Joseph W. Cutler, Serdar Tasiran, Emina Torlak, and Neha Rungta. 2024. How We Built Cedar: A Verification-Guided Approach. In *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*. https://arxiv.org/abs/2407.01688

[8] ETSI. 2025. *Analysis of Selective Disclosure and Zero-Knowledge Proofs Applied to EUDI Wallet*. Technical Report TR 119 476-1 v1.3.1. European Telecommunications Standards Institute. https://www.etsi.org/deliver/etsi_tr/119400_119499/119476/01.03.01_60/tr_119476v010301p.pdf

[9] Hossein Hafezi, Alireza Shirzad, Benedikt Bünz, and Joseph Bonneau. 2026. Transparent Dictionaries from Polynomial Commitments. In *Proceedings of the 35th USENIX Security Symposium (Cycle 1)*. USENIX Association, Berkeley, CA, USA.

[10] Vivek Haldar, Deepak Chandra, and Michael Franz. 2004. Semantic Remote Attestation: A Virtual Machine Directed Approach to Trusted Computing. In *Proceedings of the 3rd Conference on Virtual Machine Research and Technology Symposium*. https://www.usenix.org/legacy/event/vm04/tech/haldar.html

[11] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. 2009. seL4: Formal Verification of an OS Kernel. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles*. doi:10.1145/1629575.1629596

[12] Ben Laurie, Emilia Messeri, and Rob Stradling. 2021. Certificate Transparency Version 2.0. RFC 9162. doi:10.17487/RFC9162

[13] Tobias Looker, Victoria Kalos, Mike Whitehead, and Ethan Bastian. 2026. The BBS Signature Scheme. Internet-Draft draft-irtf-cfrg-bbs-signatures-10. https://datatracker.ietf.org/doc/draft-irtf-cfrg-bbs-signatures/

[14] Mark S. Miller. 2006. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. Ph. D. Dissertation. Johns Hopkins University. https://jscholarship.library.jhu.edu/bitstream/handle/1774.2/873/markm-thesis.pdf

[15] Anders Rundgren, Bret Jordan, and Samuel Erdtman. 2020. JSON Canonicalization Scheme (JCS). RFC 8785. doi:10.17487/RFC8785

[16] Ahmad-Reza Sadeghi and Christian Stüble. 2004. Property-Based Attestation for Computing Platforms: Caring About Properties, Not Mechanisms. In *Proceedings of the 2004 Workshop on New Security Paradigms (NSPW)*. doi:10.1145/1065907.1066038

[17] Reiner Sailer, Xiaolan Zhang, Trent Jaeger, and Leendert van Doorn. 2004. Design and Implementation of a TCG-Based Integrity Measurement Architecture. In *13th USENIX Security Symposium*. https://www.usenix.org/conference/13th-usenix-security-symposium/design-and-implementation-tcg-based-integrity-measurement

[18] Secure Systems Lab. 2026. Dead Simple Signing Envelope. https://github.com/secure-systems-lab/dsse

[19] Sigstore Project. 2026. Rekor: Software Supply Chain Transparency Log. https://github.com/sigstore/rekor

[20] Sigstore Project. 2026. Sigstore Security Model. https://docs.sigstore.dev/about/security/

[21] SLSA Framework. 2026. Supply-chain Levels for Software Artifacts Specification. https://slsa.dev/spec/latest/

[22] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. 2019. in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes. In *28th USENIX Security Symposium*. https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias

[23] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2025. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. In *32nd Network and Distributed System Security Symposium (NDSS)*. https://www.ndss-symposium.org/ndss2025/